1st Draft

SOFTWARE REQUIREMENTS SPECIFICATION

IT Asset Management System

ITAMS

Document Version	1.0
Status	Draft
Date	April 2026
Framework	Spring Boot + Microservices
Database	PostgreSQL
Deployment	Docker

Prepared by the ITAMS Development Team

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document describes the functional and non-functional requirements for the IT Asset Management System (ITAMS). ITAMS is a web-based enterprise application designed to track, manage, and maintain all IT hardware and software assets within an organization. It provides structured workflows for asset lifecycle management, from procurement and assignment to maintenance and retirement.

This document serves as the primary reference for the development team, stakeholders, and quality assurance personnel throughout the project lifecycle.

1.2 Scope

ITAMS will be developed as a microservices-based application using Spring Boot REST APIs, deployed via Docker. The system covers the following core functional domains:

- Asset Inventory & Tracking: full lifecycle management of IT assets
- User & Role Management: authentication, authorization, and role-based access control
- Maintenance & Repair Requests: ticket-based maintenance tracking for assets
- Reports & Analytics: dashboards and exportable reports for asset insights

The system will serve three primary user roles: Administrator, Asset Manager, and Employee. It will enforce role-based API authorization, implement Aspect-Oriented Programming (AOP) for cross-cutting concerns, and support a PostgreSQL relational database.

1.3 Intended Audience

- Development Team: backend and frontend engineers building and testing the system
- Project Stakeholders: managers and clients reviewing scope and requirements
- QA Engineers: testers validating functional and non-functional requirements
- System Architects: architects defining microservice boundaries and integrations

1.4 Definitions, Acronyms, and Abbreviations

Term	Definition
ITAMS	IT Asset Management System: the application described in this document
SRS	Software Requirements Specification
API	Application Programming Interface: RESTful endpoints exposed by Spring Boot
AOP	Aspect-Oriented Programming: used for logging, auditing, and cross-cutting concerns
JWT	JSON Web Token: used for stateless authentication and session management
RBAC	Role-Based Access Control: permissions model governing API access

Asset	Any hardware device (laptop, server, phone) or software license tracked in the system
Microservice	An independently deployable service responsible for a single business domain
Docker	Containerization platform used to package and deploy all system services
PostgreSQL	The relational database management system used for persistent data storage
FR	Functional Requirement
NFR	Non-Functional Requirement

1.5 References

- Spring Boot Official Documentation: https://spring.io/projects/spring-boot
- Spring Cloud Documentation: https://spring.io/projects/spring-cloud
- PostgreSQL 15 Documentation: https://www.postgresql.org/docs/
- Docker Official Documentation: https://docs.docker.com
- IEEE Std 830-1998: Recommended Practice for Software Requirements Specifications

2. Overall System Description

2.1 Product Perspective

ITAMS is a standalone enterprise web application built on a microservices architecture. Each business domain is implemented as an independent Spring Boot service, communicating via REST APIs and orchestrated through Spring Cloud components (Service Discovery, API Gateway, and Config Server). All services are containerized with Docker for consistent, portable deployment.

The frontend using React.js consuming the backend REST APIs. A PostgreSQL database provides persistent storage, with each microservice owning its own schema to preserve service autonomy.

2.2 Product Features (High-Level)

- Secure user registration, login, and JWT-based session management
- Full CRUD operations on IT assets with categorization and status tracking
- Asset assignment and transfer between employees
- Maintenance ticket creation, tracking, and resolution
- Role-based dashboards, reports, and data exports
- System-wide audit logging via AOP interceptors
- Containerized microservice deployment with Docker Compose

2.3 User Classes and Characteristics

Role	Profile	Primary Goals
Administrator	IT or system admin with full system access	Manage users, configure system, view all reports
Asset Manager	Manages inventory, assignments, and maintenance	Track assets, manage tickets, generate reports
Employee	End user who receives and uses IT assets	View assigned assets, submit maintenance requests

2.4 Operating Environment

- Backend: Java 17+, Spring Boot 3.x, Spring Cloud
- Database: PostgreSQL 15+
- Containerization: Docker & Docker Compose
- Authentication: Spring Security + JWT
- API Style: RESTful JSON over HTTPS
- Frontend: React.js

2.5 Design and Implementation Constraints

- All backend services must be developed as Spring Boot microservices
- All APIs must be RESTful;

- AOP must be implemented for logging and auditing cross-cutting concerns
- Every service must be Dockerized and runnable via Docker Compose
- Authentication must use JWT tokens with expiry and refresh support
- Database schema per service to maintain microservice independence

3. Functional Requirements

This section details the functional requirements organized by the four core modules of ITAMS. Each requirement is assigned a unique identifier for traceability.

*The **User & Role Management** module is split across two microservices:

- - Auth Service, which handles authentication and JWT token lifecycle
- -User Service, which handles user profile management and RBAC enforcement.*

3.1 Module 1: User & Role Management

This module handles all authentication, user administration, and role-based access control within the system. It is implemented as an independent microservice exposing secure REST endpoints protected by Spring Security and JWT.

3.1.1 User Registration & Authentication

- FR-UM-01: The system shall allow new users to register with a unique email address, full name, and password.
- FR-UM-02: Passwords must be stored using a strong hashing algorithm (BCrypt).
- FR-UM-03: Registered users shall authenticate via a POST /auth/login endpoint using email and password.
- FR-UM-04: Upon successful login, the system shall return a signed JWT access token and a refresh token.
- FR-UM-05: The system shall provide a token refresh endpoint (POST /auth/refresh) to renew expired access tokens without re-login.
- FR-UM-06: The system shall provide a logout endpoint that invalidates the active token.

3.1.2 User Management

- FR-UM-07: Administrators shall be able to create, view, update user accounts.
- FR-UM-08: Administrators shall be able to assign one or more roles to a user (Admin, Asset Manager, Employee).

- FR-UM-10: The system shall expose a GET /users endpoint (Admin only) listing all users with filtering by role and status.
- FR-UM-11: Each user shall have a profile page displaying their personal info and list of assigned assets.

3.1.3 Role-Based Access Control

- FR-UM-12: All API endpoints shall be protected and require a valid JWT token.
- FR-UM-13: The system shall enforce RBAC at the API gateway level — unauthorized role access returns HTTP 403.
- FR-UM-14: AOP-based interceptors shall log every API access attempt including the user identity, endpoint, timestamp, and result.

3.2 Module 2: Asset Inventory & Tracking

This module provides comprehensive CRUD management for all IT assets. It tracks asset metadata, current status, assignment history, and location. It is the core module of ITAMS.

3.2.1 Asset Registration

- FR-AI-01: Asset Managers and Admins shall be able to create a new asset record with the following mandatory fields: Asset Name, Asset Tag (unique ID), Category (Hardware / Software License), Brand/Model, Serial Number, Purchase Date, Purchase Cost, and Warranty Expiry Date.
- FR-AI-02: Assets shall support an optional description field and file attachments (e.g., invoices, photos).
- FR-AI-03: The system shall auto-generate a unique asset tag if one is not manually provided.
- FR-AI-04: Upon creation, the asset status shall default to 'Available'.

3.2.2 Asset Status & Lifecycle

- FR-AI-05: Each asset shall have one of the following statuses: Available, Assigned, Under Maintenance, Retired, Lost/Stolen.
- FR-AI-06: Status transitions shall be enforced — e.g., an 'Assigned' asset cannot be directly 'Retired' without first being 'Available'.
- FR-AI-07: Asset Managers shall be able to retire an asset, recording a retirement reason and date.

3.2.3 Asset Assignment & Transfer

- FR-AI-08: Asset Managers shall be able to assign an 'Available' asset to a specific Employee, recording the assignment date.
- FR-AI-09: An asset can only be assigned to one employee at a time.
- FR-AI-10: Asset Managers shall be able to revoke an assignment, returning the asset to 'Available' status.
-
- FR-AI-12: Employees shall be able to view their currently assigned assets .

3.2.4 Asset Search & Filtering

- FR-AI-13: The system shall provide a searchable asset list with filters for Category, Status, Assigned Employee, and Purchase Date range.
- FR-AI-14: Asset Managers and Admins shall be able to export the asset list to CSV.
- FR-AI-15: The system shall display a count summary by status on the inventory dashboard.

3.3 Module 3: Maintenance & Repair Requests

This module manages the lifecycle of maintenance tickets for IT assets. Employees report issues; Asset Managers triage and resolve them. AOP logging captures all status changes for audit purposes.

3.3.1 Ticket Creation

- FR-MR-01: Employees and Asset Managers shall be able to submit a maintenance request for any asset currently assigned to them.

- FR-MR-02: Each ticket shall capture: Asset Reference, Reported Issue Description, Priority (Low / Medium / High / Critical), and Reporter identity.
- FR-MR-03: Upon submission, the ticket status shall be set to 'Open' and the linked asset status shall change to 'Under Maintenance'.
- FR-MR-04: The system shall auto-assign a unique ticket ID and timestamp on creation.

3.3.2 Ticket Management

- FR-MR-05: Asset Managers shall be able to view all open and in-progress tickets with sorting by priority and submission date.
- FR-MR-06: Asset Managers shall be able to update ticket status through the following lifecycle: Open → In Progress → Resolved → Closed.
- FR-MR-07: Asset Managers shall be able to add internal notes and resolution details to a ticket.
- FR-MR-08: When a ticket is marked 'Resolved', the system shall prompt the Asset Manager to update the linked asset's status.
- FR-MR-09: Employees shall be able to view the status and progress notes of their submitted tickets.

3.3.3 Maintenance History

- FR-MR-10: The system shall retain a full maintenance history per asset, accessible to Asset Managers and Admins.
- FR-MR-11: Maintenance history shall include: ticket ID, issue description, resolution notes, technician, and time-to-resolution.
- FR-MR-12: AOP interceptors shall log every ticket status transition for audit trail purposes.

3.4 Module 4: Reports & Analytics

This module provides data-driven insights through dashboards and exportable reports. It aggregates data across the Asset Inventory and Maintenance modules to support informed decision-making by Admins and Asset Managers.

3.4.1 Dashboard

- FR-RA-01: The system shall provide a role-aware dashboard displayed upon login.
- FR-RA-02: The Admin/Asset Manager dashboard shall display: Total Assets by Status, Assets by Category, Open Maintenance Tickets by Priority,
- FR-RA-03: The Employee dashboard shall display: My Assigned Assets and My Open Tickets.
- FR-RA-04: Dashboard data shall refresh in near real-time (within 60 seconds of changes).

3.4.2 Standard Reports

- FR-RA-05: The system shall provide a Full Asset Inventory Report — all assets with current status, assigned user, and location.
- FR-RA-07: The system shall provide a Maintenance Summary Report — all tickets within a date range with resolution metrics.
- FR-RA-08: The system shall provide a Warranty Expiry Report — assets with warranties expiring within a configurable period.

3.4.3 Export & Filtering

- FR-RA-09: All reports shall support export to CSV format.
- FR-RA-10: Reports shall support filtering by date range, asset category, status, and assigned user.
- FR-RA-11: Only Admins and Asset Managers shall have access to system-wide reports.

3.5.1 Dashboard

4. Non-Functional Requirements

4.1 Security

- NFR-SEC-01: All API endpoints must require a valid JWT token. Unauthenticated requests shall return HTTP 401.
- NFR-SEC-02: Endpoints shall enforce RBAC. Requests from insufficient roles shall return HTTP 403.
- NFR-SEC-03: JWT access tokens shall expire after 15 minutes; refresh tokens shall expire after 7 days.
- NFR-SEC-04: All passwords must be hashed using BCrypt with a minimum cost factor of 10.
- NFR-SEC-05: All API communication must occur over HTTPS/TLS. Plaintext HTTP shall be disabled in production.
- NFR-SEC-06: The system shall implement protection against common OWASP Top 10 vulnerabilities including SQL Injection, XSS, and CSRF.
- NFR-SEC-07: Sensitive configuration (DB credentials, JWT secret) must be stored as Docker environment variables — never hardcoded in source code.
- NFR-SEC-08: AOP-based audit logging shall record all create, update, and delete operations with actor identity and timestamp.
- NFR-SEC-09: Failed login attempts shall be rate-limited (max 5 per minute per IP) to prevent brute-force attacks.

4.2 Performance

- NFR-PERF-01: All standard API responses shall complete within 500ms under normal load (up to 100 concurrent users).
- NFR-PERF-02: Dashboard and report endpoints may take up to 2 seconds under peak load due to aggregation complexity.
- NFR-PERF-03: Database queries on asset tables shall be optimized with appropriate indexes on frequently filtered columns (status, category, assigned_user_id).
- NFR-PERF-04: The system shall support pagination on all list endpoints (default page size: 20, max: 100).
- NFR-PERF-05: CSV export for up to 10,000 asset records shall complete within 5 seconds.

4.3 Scalability

- NFR-SCL-01: The microservices architecture shall allow each service (User, Asset, Maintenance, Reports) to be scaled independently without affecting other services.
- NFR-SCL-02: Spring Cloud Service Discovery (Eureka) shall be used to enable dynamic service registration and load balancing.
- NFR-SCL-03: The system shall support horizontal scaling of stateless services by running multiple container instances behind the API Gateway.
- NFR-SCL-04: The PostgreSQL database shall be configured with connection pooling (HikariCP) to handle concurrent connections efficiently.
- NFR-SCL-05: Docker Compose configuration shall define resource limits per service to prevent single-service resource exhaustion.

4.4 Availability & Reliability

- NFR-AVL-01: The system shall target 99.5% uptime during business hours.
- NFR-AVL-02: All microservices shall implement health-check endpoints (GET /actuator/health) for monitoring.
- NFR-AVL-03: Graceful shutdown must be configured — in-flight requests shall complete before a service terminates.
- NFR-AVL-04: The system shall implement circuit-breaker patterns (Spring Cloud Circuit Breaker / Resilience4j) to prevent cascade failures.

4.5 Maintainability

- NFR-MNT-01: All services shall follow a consistent layered architecture: Controller → Service → Repository.
- NFR-MNT-02: AOP shall be used for all cross-cutting concerns (logging, auditing) to keep business logic clean.
- NFR-MNT-03: Each microservice shall have its own Maven/Gradle module and independent Docker image.
- NFR-MNT-04: Code shall follow Java naming conventions and include Javadoc for all public APIs.

5. User Roles & Permissions Matrix

The following matrix defines access control for all system features across the three user roles. Access is enforced at the API level using Spring Security and JWT-based RBAC. AOP interceptors audit all sensitive operations.

Feature / Permission	Admin	Asset Manager	Employee
USER MANAGEMENT			
Register / Create user account	✓ Full	✗	Self only
View user list	✓ Full	✗	✗
Edit user details	✓ Full	✗	Self only
Assign / change user roles	✓ Full	✗	✗
Deactivate user account	✓ Full	✗	✗
View own profile	✓	✓	✓
ASSET INVENTORY			
Create new asset	✓	✓	✗
View all assets	✓	✓	✗
View own assigned assets	✓	✓	✓
Edit asset details	✓	✓	✗
Assign asset to employee	✓	✓	✗
Revoke asset assignment	✓	✓	✗
Retire / decommission asset	✓	✓	✗
Export asset list (CSV)	✓	✓	✗
MAINTENANCE & REPAIR			
Submit maintenance request	✓	✓	✓ (own assets)
View all maintenance tickets	✓	✓	✗
View own submitted tickets	✓	✓	✓
Update ticket status	✓	✓	✗
Add notes/resolution to ticket	✓	✓	✗
View full maintenance history	✓	✓	✗
REPORTS & ANALYTICS			

Feature / Permission	Admin	Asset Manager	Employee
View admin dashboard	✓	✓	✗
View employee dashboard	✓	✓	✓
Run system-wide reports	✓	✓	✗
Run personal asset report	✓	✓	✓
Export reports (CSV)	✓	✓	✗
View audit logs	✓	✗	✗

5.1 Role Descriptions

Administrator

The Administrator has unrestricted access to all system modules and data. This role is responsible for managing user accounts and roles, system configuration, and audit log review. Typically held by the IT department head or system administrator.

Asset Manager

The Asset Manager has operational control over the asset inventory and maintenance workflows. This role can create and manage assets, assign them to employees, process maintenance tickets, and generate organizational reports. It cannot manage user accounts or view audit logs.

Employee

The Employee role has limited, read-focused access scoped to their own data. Employees can view their assigned assets, submit maintenance requests for issues they encounter, and track the status of their tickets. They cannot view organizational-level data or manage any system entities.

6. System Architecture Overview

6.1 Microservices Architecture

ITAMS is decomposed into four domain microservices, each responsible for a single bounded context, plus three Spring Cloud infrastructure services:

Domain Microservices

- Auth Service - handles registration, login, JWT issuance/validation, token refresh, and logout.
- User Service - handles user profile management, role assignment, and user directory.
- Asset Service - manages asset CRUD, lifecycle states, assignment, and transfer operations
- Maintenance Service - manages ticket creation, workflow, notes, and resolution tracking
- Reports Service - aggregates data from other services and generates dashboard metrics and exports

Infrastructure Services

- API Gateway (Spring Cloud Gateway) — single entry point for all client requests; handles routing, JWT validation, and rate limiting
- Service Discovery (Eureka Server) — registers and discovers all running service instances
- Config Server (Spring Cloud Config) — centralizes externalized configuration for all services

6.2 Database Design

Each microservice owns and manages its own PostgreSQL schema (Database-per-Service pattern). Inter-service data access is achieved via API calls, not shared database joins. Key tables per service:

- Auth Service: users, user_credentials, refresh_tokens
- User Service: user_profiles, roles, user_roles, departments
- Asset Service: assets, asset_categories, asset_assignments, assignment_history
- Maintenance Service: tickets, ticket_notes, ticket_status_history
- Reports Service: (read-only views / aggregation queries via internal API calls)

6.3 Cross-Cutting Concerns (AOP)

Spring AOP is used to implement the following cross-cutting concerns without polluting business logic:

- Audit Logging Aspect — intercepts all @Service method calls annotated with @Auditable and logs actor, method, arguments, and timestamp to the audit_logs table
- Performance Monitoring Aspect — measures execution time for all controller endpoints; logs warnings for responses exceeding 1 second
- Exception Handling Aspect — centralized exception mapping via @RestControllerAdvice to consistent API error responses

7. Constraints & Assumptions

7.1 Technical Constraints

- The backend must use Spring Boot 3.x as the primary framework.
- All services must expose RESTful JSON APIs.
- AOP must be demonstrably used for at least one cross-cutting concern (logging or auditing).
- All services must be packaged as Docker images and runnable via a single Docker Compose file.
- Spring Cloud (Gateway + Eureka) must be integrated.
- Database must be PostgreSQL with each service using a separate schema or database.

Non-Functional Requirements

Non-Functional Requirements

IT Asset Management System (ITAMS)

This section specifies the non-functional requirements that define system quality attributes including performance, security, scalability, maintainability, usability, and deployment constraints.

NFR-1: Performance

ID	Requirement	Acceptance Metric
NFR-P-01	API response time for CRUD operations shall be < 500ms under normal load	Avg response < 500ms, P95 < 1s
NFR-P-02	Dashboard shall load within 2 seconds on first visit	LCP < 2s
NFR-P-03	System shall support at least 100 concurrent users without degradation	Load test with 100 users
NFR-P-04	Database queries shall complete within 200ms for indexed operations	Query execution plan < 200ms
NFR-P-05	Dashboard data shall refresh within 60 seconds of data changes	Polling interval ≤ 60s

NFR-2: Security

ID	Requirement	Acceptance Metric
NFR-S-01	All passwords shall be hashed using BCrypt with strength ≥ 10	BCrypt rounds ≥ 10
NFR-S-02	All API endpoints shall require valid JWT authentication	401 on missing/invalid token
NFR-S-03	JWT tokens shall expire after 1 hour; refresh tokens after 7 days	Token TTL enforced
NFR-S-04	RBAC shall be enforced at API Gateway level — unauthorized access returns 403	Gateway filter validation
NFR-S-05	All inter-service communication shall occur within a private Docker network	No public port exposure
NFR-S-06	SQL injection and XSS shall be prevented via parameterized queries and input sanitization	OWASP Top 10 compliance
NFR-S-07	AOP interceptors shall log every API access attempt with user identity and timestamp	Audit log entries per request

NFR-3: Scalability & Availability

ID	Requirement	Acceptance Metric
NFR-A-01	Each microservice shall be independently deployable and horizontally scalable	Docker Compose scale command
NFR-A-02	Service discovery via Eureka shall support dynamic instance registration	Eureka dashboard verification
NFR-A-03	API Gateway shall route traffic using load balancing across service instances	Round-robin routing test
NFR-A-04	System shall recover from single service failure without total system downtime	Kill one service, others remain operational

NFR-4: Maintainability

ID	Requirement	Acceptance Metric
NFR-M-01	Codebase shall follow layered architecture (Controller → Service → Repository)	Code review checklist
NFR-M-02	All REST APIs shall be documented with Swagger/OpenAPI annotations	Swagger UI accessible per service
NFR-M-03	AOP shall be used for cross-cutting concerns (logging, performance monitoring)	No logging code in business logic
NFR-M-04	Database migrations shall use versioned scripts (Flyway or Liquibase)	Migration history table
NFR-M-05	Each microservice shall have its own independent PostgreSQL database	Separate DB per docker-compose service

NFR-5: Usability

ID	Requirement	Acceptance Metric
NFR-U-01	UI shall be responsive and functional on desktop (1024px+) and tablet (768px+)	Visual regression test on breakpoints
NFR-U-02	All form inputs shall provide validation feedback before submission	Client-side validation on required fields
NFR-U-03	Navigation shall be role-aware — users only see pages permitted by their role	Menu items filtered by role
NFR-U-04	System shall provide clear error messages for failed operations	Error toast/alert with actionable text

NFR-6: Portability & Deployment

ID	Requirement	Acceptance Metric
NFR-D-01	All services shall be containerized using Docker with multi-stage builds	Dockerfile per service
NFR-D-02	Full system shall be deployable via a single docker-compose up command	docker-compose.yml orchestration
NFR-D-03	Environment-specific configuration shall use environment variables, not hardcoded values	.env file + Spring profiles
NFR-D-04	Frontend shall be served as static files via Nginx or similar in production	Vite build + nginx container